

Project: Programming Language C++
Document Number: WG21/P0845R0
Date: 2017-10-13
Author: Detlef Vollmann, dv@vollmann.ch
Target audience: LEWG, LWG

P0845R0: Common Subset of C++03 and C++17: Binders

Introduction

There are many reasons to have source code that needs to compile across a wide range of C and C++ compilers. There are even more reasons to have source code that needs to compile across a wide range of different C++ compilers and versions. There are still many environments where only C++03 is supported. However, with C++17 it turns out that while the common subset of C and C++ is pretty stable, regardless of the version of C++, the common subset between different C++ versions is shrinking significantly. It would be bad to tell people to use the common subset of C and C++ if they want API stability.

All removals of features in a new version of C++ diminishes the size of the common subset between the new version and the older ones. The removal of the old C++98/03 function binders is an exceptionally bad example of feature removal.

Some of the reasons include:

- The binders were widely used.
- The removal is optional, without even a feature macro.
- There's no syntactically and semantically equivalent replacement.

Easy solution

Put the binders back in.

This is the easiest solution and would solve all problems.

Complicated solution

If the binders are not put back in, for source code to work with old and with new compilers several changes are required:

1. Provide feature macros.
2. Don't reserve the names if the binders are not provided.
3. Specify the return types of bind functions or guarantee that `decltype` works.

For change 1, it would also be possible to disallow still providing the binders (i.e. make the removal non-optional), but I don't think anybody would like this option (I don't).

Change 2 is not strictly required, but it doesn't make much sense to reserve these names if the features aren't there and it makes the life of the programmer somewhat easier if they don't have to change the name globally.

Making dec1type work in change 3 is the topic of a separate issue ([LWG 3023](#)).